

ASIC 期末课程设计:

人体反应速率检测系统

刘恒雨 23300756003

2026 年 05 月 23 日

復旦大學



目录

1 报告和演示视频获取地址	3
2 设计规则	3
2.1 设计要求	3
2.2 设计思路	3
2.2.1 设计平台	3
2.2.2 设计流程图	3
3 设计实现	5
3.1 顶层架构与模块划分	5
3.2 各模块和验证	6
3.2.1 状态机 <code>state.sv</code>	6
3.2.2 按键消抖 <code>debouncer.sv</code>	8
3.2.3 通用计数器 <code>generic_counter.sv</code>	9
3.2.4 通用倒计时器 <code>generic_countdown_counter.sv</code>	10
3.2.5 线性反馈移位寄存器 <code>lfsr.sv</code>	10
3.2.6 VGA 时序生成 <code>timing_counter.sv</code>	11
3.2.7 TMDS 编码与串化	12
3.2.7.1 TMDS 编码器 <code>tmds_encoder.sv</code>	12
3.2.7.2 TMDS 串化器 <code>tmds_serdes.sv</code>	12
3.2.8 流水线延迟对齐 <code>pipe.sv</code>	12
3.2.9 显示流水线时序	13
3.2.10 字符缩放与定位	13
3.2.11 VGA 与 TMDS 总线接口	14
3.2.12 BRAM 模块 <code>ascii_rom.sv</code>	14
3.2.13 滑动平均滤波 <code>latency_avg</code>	14
3.2.14 作弊检测 (SPAM)	15
4 仿真与测试	17
4.1 Rust VGA 仿真器	17
4.2 Rust 自动化测试	18
4.3 justfile 工作流	19
4.4 测试波形图	20
4.5 FPGA 板介绍	21
4.6 综合与实现	21
4.6.1 综合条件	21
4.6.2 资源利用率	21
4.6.3 时序结果	22
4.6.4 功耗	22
4.7 芯片物理实现	23
4.8 上板结果展示	25
4.9 设计总结	26
4.10 个人体会	27

一 报告和演示视频获取地址

项目地址

```
https://github.com/B83C/asci_final_fpga_proj
```

视频地址

```
https://github.com/B83C/asci_final_fpga_proj/raw/refs/heads/main/output.mp4
```

该文档地址

```
https://github.com/B83C/asci_final_fpga_proj/blob/main/report.pdf
```

二 设计规则

2.1 设计要求

设计一套用于测试人体反应速率的系统。用户需要根据系统状态变化做出相对应的动作。系统通过计算用户的反应时间得到其反应速率。

2.2 设计思路

2.2.1 设计平台

本次采用了 ZYNQ-Z7020 开发板（鹿小班），搭载 HDMI 输出接口。系统主时钟 50 MHz，经过 PLL（通过 Vivado tcl 脚本生成 clk_wiz_0 IP）产生 75 MHz 像素时钟和 375 MHz 串化时钟用于 HDMI TMDS 输出。

开发工具链完全采用开源方案：

- 仿真与验证：Verilator + Rust + marlin 框架，实现自定义 VGA 像素级仿真
- 综合与实现：Vivado 2024
- 下载：openFPGALoader (just upload)
- 物理实现：Vivado + librelane（含 DRC、LVS、PEX）

2.2.2 设计流程图

本系统主要用于测试用户的反应速率。系统运行过程中，会随机改变当前状态，并要求用户在限定时间内做出对应操作。系统通过记录状态变化到用户输入之间的时间差，计算用户的反应时间，并最终得到其反应速率。

系统整体运行流程如图所示，主要包括以下几个阶段：

1. 系统初始化（时钟锁定、PLL 稳定）
2. 等待用户开始（IDLE 状态）
3. 随机延时阶段（LFSR 产生 2-5 s 随机延迟）

4. 状态触发阶段（屏幕显示变化，开始计时）
5. 用户响应检测（按键消抖 → 状态转移）
6. 防作弊检测（提前按键 → SPAM 状态）
7. 反应时间计算与平均
8. 结果显示 HDMI 输出
9. 系统复位并重新开始

系统上电后，首先完成时钟锁定与复位计数，待 PLL 输出稳定后释放复位，进入待机状态（IDLE）。

当用户按下开始按键后，系统不会立即进入测试，而是先进入随机等待阶段（INIT → TRIGGERED，使用 LFSR 产生 2-5 s 随机延迟）。该设计用于避免用户通过预判时间提前做出动作，从而提高测试真实性。

随机等待结束后，系统进入 MEASURING 状态，同时开始计时，屏幕显示提示文字。

用户需要在状态变化后尽快按下响应按键。系统通过 debouncer 链对按键进行消抖处理，当检测到有效输入后停止计时，记录反应时间，并计算最近 3 次测量的滑动平均值。

若用户在测量开始前提前按键（TRIGGERED 状态下的连续按键），系统判定为作弊，进入 SPAM 状态并重置。

最终系统将反应时间以 BCD 码形式显示在屏幕上，并等待下一次测试。

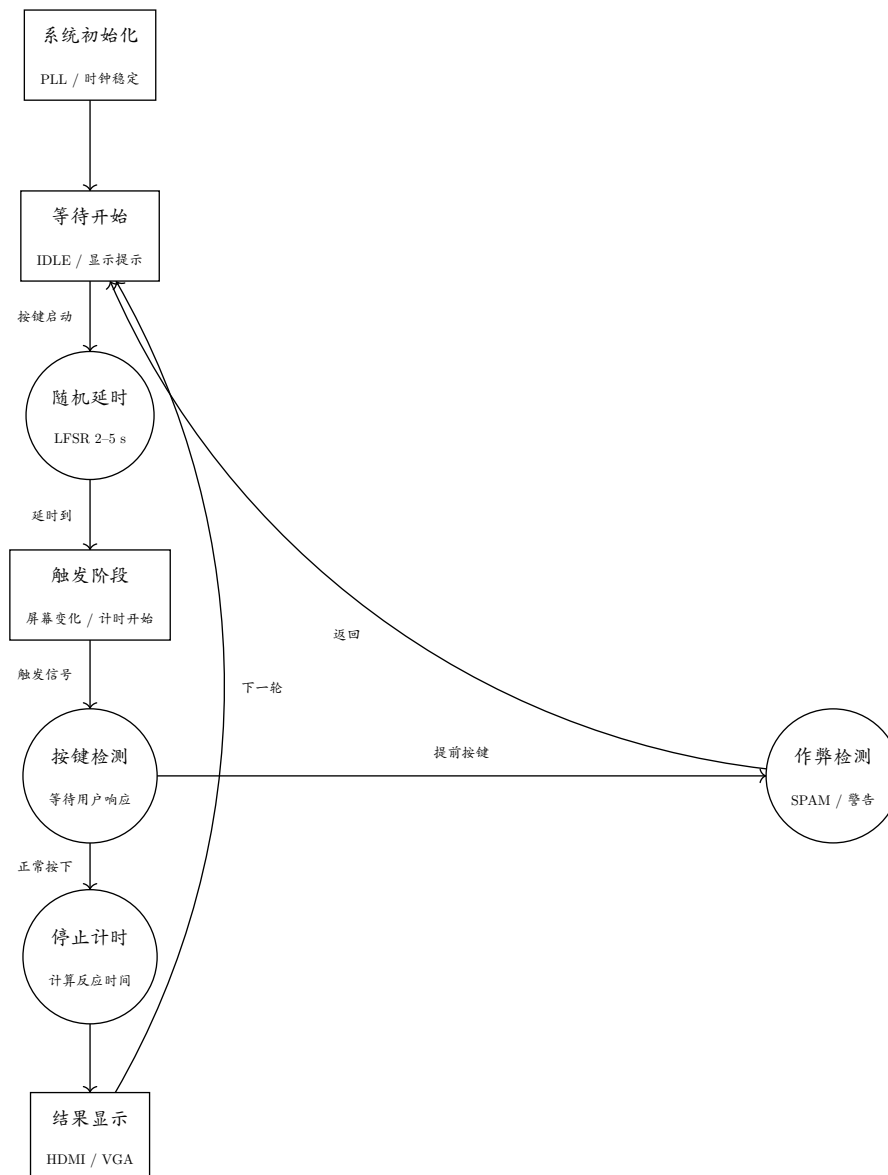


图 1 系统设计流程图（自顶向下）

三 设计实现

3.1 顶层架构与模块划分

整个系统由 *top.sv* 中的 *top_vga* 模块（VGA 显示 + 控制逻辑）、*top_dvi* 模块（TMDS 编码与串化）、*top* 模块（时钟管理 + 顶层连线）三级组成。

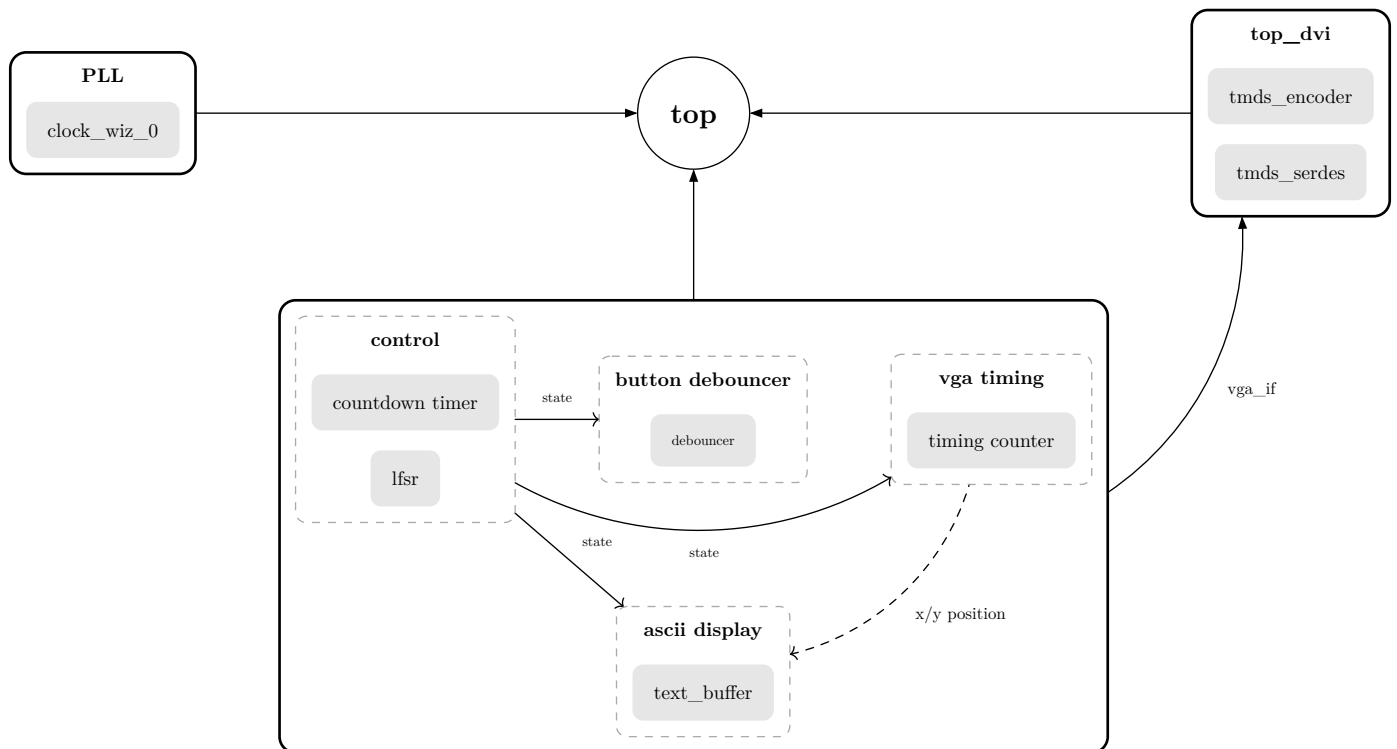


图 2 系统模块层次与分组关系 (虚线框表示逻辑分组)

top_vga 内部包含以下子模块：

- 计时相关：generic_counter (毫秒脉冲生成 + 测量计数)、generic_countdown_counter (随机倒计时)
- 随机数：lfsr (线性反馈移位寄存器，产生 2-5 s 随机延时)
- 状态机：state (6 状态：IDLE / INIT / TRIGGERED / MEASURING / MEASURED / SPAM)
- 显示：timing_counter $\times$ 2 (行/场同步时序)、ram (ASCII 字形 ROM)、pipe (流水线延迟对齐)
- 按键输入：debouncer 级联链

3.2 各模块和验证

3.2.1 状态机 state.sv

系统核心状态机，共 6 个状态，ready 信号触发状态转移，hold 信号实现握手协议防止同一脉冲多次触发。halt 信号用于作弊检测 (TRIGGERED 下将跳转到 SPAM)。

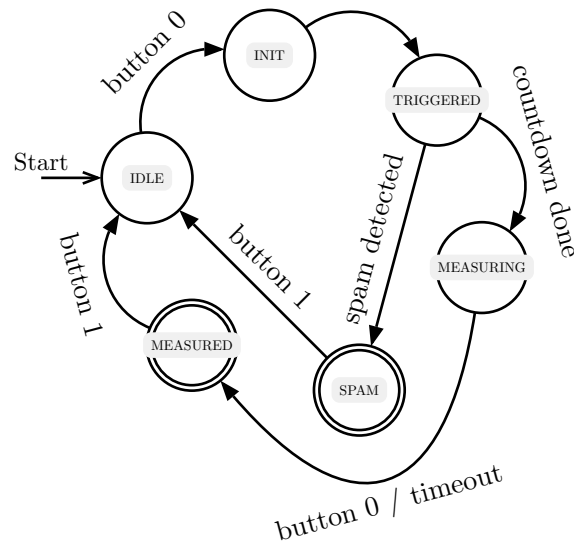


图 3 状态转移图, 其中 button x 为经过滤的输入信号

整体状态转换图如 图 3 所示。实际上, INIT 状态的作用是初始化系统所需的参数, 比如从 LFSR 模块获得随机的延迟时间。TRIGGERED 状态下等待倒计时。MEASURING 状态下计算用户的反应时间, 计算单位为毫秒。最终, 停止在 MEASURED 状态, 并输出反应延迟时间。期间, 如果 TRIGGERED 状态下用户反复输入, 则会判定为 SPAM, 并终止测量程序。此时, 不会计入延迟时间。

当然, 为了确保长按按键不会导致系统不停的转换状态, 实际上加入了额外的状态 hold 用来表示上一级的输入是否还保持 active。这样设计的一个好处是:

1. 简化模块输入
2. 让状态转换变成电平敏感而非边沿敏感, 这样对硬件时序会比较友好。

```

MEASURING: begin
  if (ready && !hold) begin
    state <= MEASURED;
    hold <= 1;
  end
end
  
```

代码 1 MEASURING 到 MEASURED 状态转换流程

```

if (hold && !ready) begin
  hold <= 0;
end
  
```

代码 2 hold 归零逻辑

模块定义如下:

```

module state (
    input clk,
    input ready,
    input halt,
    input [1:0] bi,
    output state_t sys_state
);

```

验证 :SystemVerilog testbench 在 tests/tb_state.sv 中覆盖了正常循环、长按保持、快速脉冲、作弊 halt、halt 在非 TRIGGERED 下被忽略共 5 个场景。使用 Verilator 运行：

```
just verilator state
```

3.2.2 按键消抖 debouncer.sv

采用移位寄存器 + 状态机实现按键消抖。通过深度可配置的移位缓冲器存储采样值，当连续采样均为同一电平时输出翻转。

```

module debouncer #(
    parameter unsigned DEPTH = 64,
    parameter unsigned INVERT = 1
)(
    input clk,
    input rstn,
    input raw_input,
    output reg debounced_output
);

```

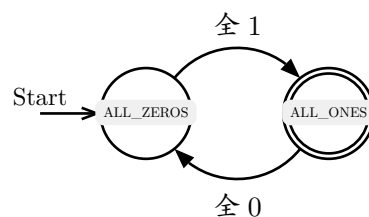


图 4 debouncer 状态转移图, 其中 button x 为经过滤的输入信号

top_vga 中使用了两级级联的 16 级深度消抖链：

```

generate
    genvar j;
    for (j = 0; j < 2; j++) begin : gen_debouncer
        localparam unsigned STAGES = 16;

```

```

wire stage[STAGES];
genvar i;
for (i = 0; i < STAGES; i++) begin : gen_debouncer_stage
    wire out;
    debouncer #(
        .DEPTH(2),
        .INVERT(i == 0)
    ) db (
        .rstn(rstn),
        .clk(clk),
        .raw_input(i == 0 ? buttons[j] : stage[i-1]),
        .debounced_output(out)
    );
    if (i == STAGES - 1) begin : gen_assign_to_final
        assign bi[j] = out;
    end else begin : gen_assign_to_inner_stages
        assign stage[i] = out;
    end
end
end
endgenerate

```

验证：SystemVerilog testbench tests/tb_debouncer.sv 覆盖了按键按下、保持、释放、毛刺抑制、复位 5 种场景。另有 Rust 测试 tests/test2.rs 使用 marlin_test 框架验证 debouncer2 的 triggered 脉冲。

3.2.3 通用计数器 generic_counter.sv

可配置计数上限、单次模式（ONESHOT），输出当前计数值和结束标志。

```

module generic_counter #(
    parameter unsigned MAX = 1280,
    parameter unsigned ONESHOT = 0,
    parameter unsigned N = $clog2(MAX)
) (
    input clk,
    input rstn,
    input en,

```

```
output [N - 1:0] x,

output ending
);
```

验证：Rust 测试 tests/simple_test.rs 含 test_counter 和 test_counter_unen，验证基本计数和 enable 门控。

```
just test test_counter
```

3.2.4 通用倒计时器 generic_countdown_counter.sv

与 generic_counter 配合使用，用于生成随机等待倒计时。

```
module generic_countdown_counter #(
    parameter unsigned MAX = 65536,
    parameter unsigned N = $clog2(MAX)
)(
    input clk,
    input rstn,
    input en,

    input [N - 1:0] load_val,
    input load,

    output done
);
```

验证：tests/simple_test.rs 中的 test_countdown 加载初值后验证逐周期递减至零。

3.2.5 线性反馈移位寄存器 lfsr.sv

16-bit LFSR，反馈多项式 $x^{16} + x^{13} + x^{12} + x^{10} + 1$ 。通过参数 MIN / MAX 限定输出范围。

```
module lfsr #(
    parameter unsigned MIN = 100,
    parameter unsigned MAX = 65536,
    parameter unsigned SEED = 16'hFADE,
    localparam unsigned N = $clog2(MAX)
```

```

)(
    input clk,
    input rstn,
    input en,

    output [N - 1:0] out
);

```

验证：tests/simple_test.rs 中 test_vga 验证 LFSR 输出在 16 个桶内大致均匀分布。

3.2.6 VGA 时序生成 timing_counter.sv

生成 VGA 行/场同步信号和有效显示区域标志。参数化前端、同步脉冲、后沿宽度，支持任意分辨率。

```

module timing_counter #(
    parameter unsigned DISP = 640,
    parameter unsigned FP = 16,
    parameter unsigned SW = 96,
    parameter unsigned BP = 48,
    parameter unsigned N = 12
)(
    input pixclk,
    input rstn,
    input en,

    output reg sync,

    output [N - 1:0] x,

    output active,
    output ending
);

```

当前配置为 1280×720@60Hz（75 MHz 像素时钟）：

- 行：显示 1280 + 前端 110 + 同步 40 + 后沿 220 = 1650
- 场：显示 720 + 前端 5 + 同步 5 + 后沿 20 = 750

验证：tests/simple_test.rs 中 test_vga_signals 运行 200000 周期检查时序，或直接运行仿真程序使用宏四边窗口观察。

3.2.7 TMDS 编码与串化

3.2.7.1 TMDS 编码器 tmds_encoder.sv

实现 DVI 标准的 TMDS 编码算法，包含异或/异或非模式选择、DC 平衡累加、控制周期编码。

```
module tmds_encoder (
    input clk,
    input [7:0] video_data,
    input [1:0] control_data,
    input video_data_enable,
    output reg [9:0] TMDS = 0
);
```

3.2.7.2 TMDS 串化器 tmds_serdes.sv

使用 Xilinx 7 系列原语 OSERDESE2 (MASTER + SLAVE 级联) 实现 10:1 串化。OBUFDS 输出差分 TMDS 信号。

```
module tmds_serdes (
    input pixclk,
    input serdes_clk,
    input rstn,
    input [9:0] tmds_p,
    output tmds_tx_serial_p,
    output tmds_tx_serial_n
);
```

3.2.8 流水线延迟对齐 pipe.sv

通用流水线寄存器，用于对齐因 BRAM 读取、TMDS 编码等多周期延迟导致的信号漂移。

```
module pipe #(
    parameter unsigned STAGES = 1,
    parameter unsigned M = 1
)(
    input clk,
    input rstn,
    input en,

    input [M - 1:0] x,
    output [M - 1:0] y
);
```


3.2.9 显示流水线时序

以下时序图展示了像素数据从坐标输入到 HDMI (VGA) 输出的完整流水线延迟 (75 MHz):

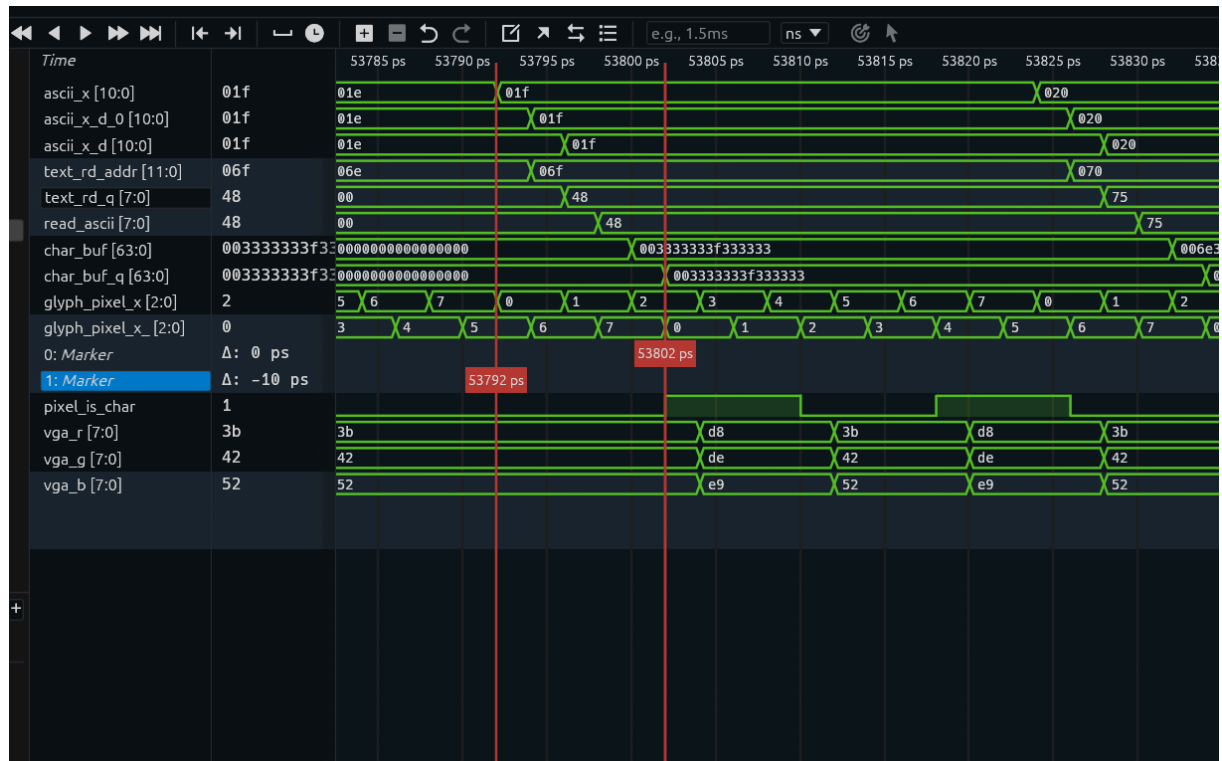


图 5 VGA 部分数据流水线波形图, 6 周期

如上所示, 为了提高时序性能, 本人采用 6 级流水线 (可优化)。一开始有计数器产生对应输出 ascii 地址 `ascii_x`。过程中 `ascii_x` 从 `text_buffer`, 读取字体的像素 `char_buf`, 然后最终转译为 `pixel_is_char`, 表示该处是否显示字体的颜色。

3.2.10 字符缩放与定位

系统通过三级级联计数器实现像素坐标到字符坐标的转换, 同时利用 `SCALE_X / SCALE_Y` 参数实现字符缩放:

- `sx / sy`: 像素级缩放计数器, 周期为 `SCALE_X / SCALE_Y`
- `cx / cy`: 字形像素坐标, 遍历字符位图的每个像素列/行 (`C_H / C_V`)
- `xp / yp`: ASCII 字符坐标, 定位到文本缓冲区中的字符位置

以水平方向为例, `sx` 在每个像素时钟累加, 每 `SCALE_X` 个时钟产生一个进位使 `cx` 递增, 扫描字形的一个像素列; `cx` 走过整个字符宽度 `C_H` 后进位使 `xp` 递增, 移动到下一个字符位置。垂直方向同理, 形成嵌套扫描关系: $\text{pixel_x} = \text{ascii_x} * C_H * \text{SCALE_X} + \text{glyph_pixel_x} * \text{SCALE_X} + \text{sx}$ 。

当前参数 ($H=1280, V=720, C_H=8, C_V=8, \text{SCALE_X}=2, \text{SCALE_Y}=2$) 下, 屏幕划分为 80×45 的字符网格 ($\text{HCC} = H / (C_H * \text{SCALE_X}), \text{VCC} = V / (C_V * \text{SCALE_Y})$), 每个字符占 16×16 像素。字形 ROM (`ascii_rom.sv`) 存储 ASCII 码点对应的 8×8 位图, 经 `SCALE_X / SCALE_Y` 放大后显示。

3.2.11 VGA 与 TMDS 总线接口

```
interface vga_if;
    logic hsync, hblank, vsync, vblank, active, fsync;
    logic [7:0] r, g, b;

    modport consumer(input hsync, hblank, vsync, vblank, active, fsync, r, g, b);
    modport producer(output hsync, hblank, vsync, vblank, active, fsync, r, g, b);
endinterface
```

```
interface tmds_bus_if;
    logic tmds_tx_clk_n;
    logic tmds_tx_clk_p;

    logic [2:0] tmds_tx_data_p;
    logic [2:0] tmds_tx_data_n;
endinterface
```

3.2.12 BRAM 模块 ascii_rom.sv

```
module ascii_rom #(
    parameter int WIDTH = 16,
    parameter int DEPTH = 8,
    parameter int ADDR_BITS = $clog2(DEPTH)
)(
    input clk,
    input [ADDR_BITS - 1:0] addr,
    output logic [WIDTH - 1:0] data
);
```

3.2.13 滑动平均滤波 latency_avg

在 top_vga 中实现了深度为 3 的滑动平均滤波，用于平滑反应延迟测量值。模块端口定义如下：

每次测量完成后，新值进入 latency_buffer 并淘汰最旧值。当缓冲区未填满时直接使用当前值填满；填满后使用递推公式：

$$\text{avg} \leftarrow \text{avg} + \frac{\text{new}}{3} - \frac{\text{oldest}}{3}$$

具体实现位于 MEASURING 状态分支：

```

MEASURING: begin
    if (ready && !ms_done) begin
        measured_latency <= ms_elapsed;
        latency_buffer[0] <= ms_elapsed;
        latency_buffer[1] <= latency_buffer[0];
        latency_buffer[2] <= latency_buffer[1];
        if (latency_buffer[HIST_DEPTH-1] == 0) begin
            latency_buffer[1] <= ms_elapsed;
            latency_buffer[2] <= ms_elapsed;
            latency_avg <= ms_elapsed;
        end else begin
            latency_avg <= latency_avg + ((ms_elapsed) / HIST_DEPTH) -
                ((latency_buffer[HIST_DEPTH-1]) / HIST_DEPTH);
        end
    end
end
end

```

3.2.14 作弊检测 (SPAM)

为防止用户在随机延时阶段 (TRIGGERED 状态) 提前按键作弊, 系统先对按键 bi[0] 做上升沿检测, 再对边沿脉冲计数:

```

logic bi_0_d1;
logic bi_0_rise;
always @(posedge clk) bi_0_d1 <= bi[0];
assign bi_0_rise = bi[0] && !bi_0_d1;

logic [1:0] spam_count;
always @(posedge clk) begin
    if (!rstn || sys_state == IDLE) begin
        spam_count <= 0;
    end else if (sys_state == TRIGGERED && bi_0_rise) begin
        spam_count <= spam_count + 1;
    end
end
end

```

检测原理: bi_0_d1 通过一级寄存器提取 bi[0] 的上一周期值, bi_0_rise = bi[0] && !bi_0_d1 得到上升沿脉冲, 确保每次按键只产生一个计数。spam_count 仅在 TRIGGERED 状态下且检测到上升沿时递增, 复位或在 IDLE 状态时清零。当计数达到阈值时 halt 信号拉高, 状态机立即跳转至 SPAM:

```

TRIGGERED: begin
    if (halt) begin
        state <= SPAM;
        hold <= 0;
    end else if (ready && !hold) begin
        state <= MEASURING;
        hold <= 1;
    end
end
end

```

进入 SPAM 后，通过 reset_counter 复位相关计数器，同时显示流水线输出作弊警告：

```

logic [7:0] buffer[10];
always_comb begin
    case (sys_state)
        IDLE: buffer = {>>8{" IDLE"}};
        INIT: buffer = {>>8{" INIT"}};
        TRIGGERED: buffer = {>>8{" WAITING"}};
        MEASURING: buffer = {>>8{" MEASURING"}};
        MEASURED: buffer = {>>8{" MEASURED"}};
        SPAM: buffer = {>>8{" SPAMMED!"}};
        default: buffer = {>>8{"UNDEFINED"}};
    endcase
end

```

```

logic [7:0] prompt[20];
always_comb begin
    if (sys_state == SPAM) prompt = {>>8{"spam: dont cheat "}};
    else if (sys_state == MEASURING) prompt = {>>8{" >> press now << "}};
    else if (timed_out) prompt = {>>8{" timed out! "}};
    else prompt = {>>8{" "}};
end

```

验证：状态机测试 tests/tb_state.sv 覆盖了 halt 触发 SPAM 的场景，验证 TRIGGERED 下 halt 有效跳转至 SPAM，且非 TRIGGERED 下 halt 被忽略。

四 仿真与测试

4.1 Rust VGA 仿真器

本项目开发了一套基于 Rust + macroquad + Verilator 的自定义 VGA 仿真器 (src/main.rs)，可以在 PC 窗口上实时渲染 FPGA 像素输出并接收键盘输入模拟按键。

```
module top_simulation (  
    input clk,  
    input rstn,  
  
    input [1:0] buttons,  
  
    output reg [1:0] leds,  
  
    output hsync,  
    output hblank,  
    output vsync,  
    output vblank,  
    output active,  
    output fsync,  
  
    output logic [7:0] r,  
    output logic [7:0] g,  
    output logic [7:0] b  
);
```

```
use marlin::{verilator::tracing::Trace, verilog::prelude::*};  
  
#[verilog(src = "src/top_simulation.sv", name = "top_simulation")]  
pub struct TopVga;  
  
impl TopVga<'_> {  
    pub fn tick(&mut self, trace: &mut Option<Trace<'_>>, timestamp: &mut u64) {  
        self.clk = 0;  
        self.eval();  
        if let Some(trace) = trace {  
            *timestamp += 1;  
            trace.dump(*timestamp);  
        }  
    }  
}
```

```

self.clk = 1;
self.eval();
if let Some(trace) = trace {
    *timestamp += 1;
    trace.dump(*timestamp);
}
}
}
}

```

仿真线程通过 VerilatorRuntime 加载 SystemVerilog 源文件，创建 TopVga 模型，逐像素扫描 hsync / vsync 同步信号，将 r / g / b 值写入帧缓冲区。宏四边窗口以 1280×720 分辨率实时显示 VGA 画面，键盘 Space / T 键映射为 FPGA 按钮输入。

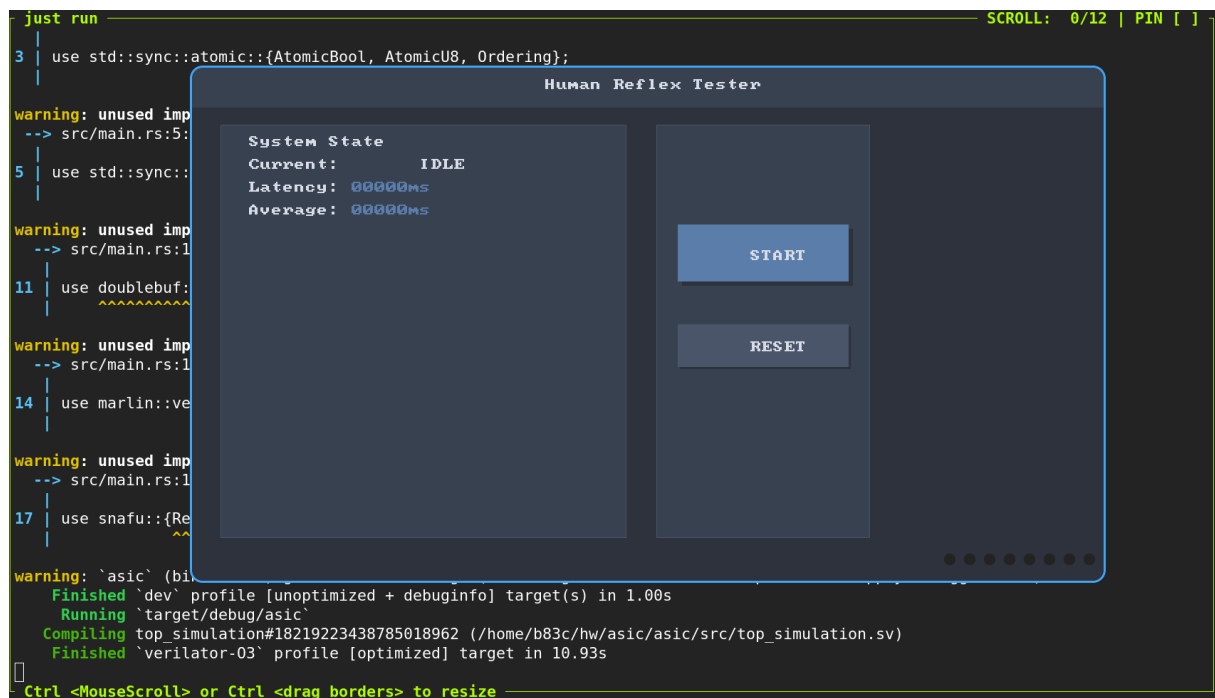


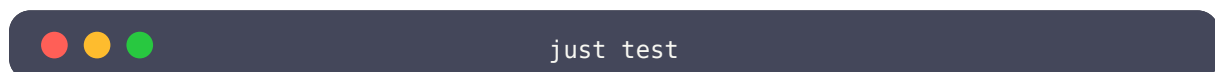
图 6 仿真图形化界面

4.2 Rust 自动化测试

使用 marlin_test 框架进行单元级验证。每个测试用 `#[verilog(src = "...", name = "...")]` 声明待测模块，`#[marlin_verilog_test]` 自动编译并用 Verilator 运行。

测试源码位于 `tests/simple_test.rs` 和 `tests/test2.rs`，包含 LFSR 分布测试、计数器启停测试、倒计时器测试、VGA 信号测试、消抖器边缘用例测试（按下/保持/释放/毛刺/复位）。

运行命令：



```
just          # 全部测试
just          # 单个测试
just          # LFSR 分布测试
```

此外还有 SystemVerilog testbench :

```
just          # 状态机测试
just          # 消抖测试
```

4.3 justfile 工作流

所有常见操作通过 just 命令管理 :

```
just          # 运行 Rust 自动化测试
just          # 启动 VGA 仿真窗口
just          # 仿真 + 导出 FST 波形
just          # Vivado 综合
just          # openFPGALoader 下载到 FPGA
just          # 打开波形查看
```

4.4 测试波形图

由于测试波形太多，这里只展示一部分波形图

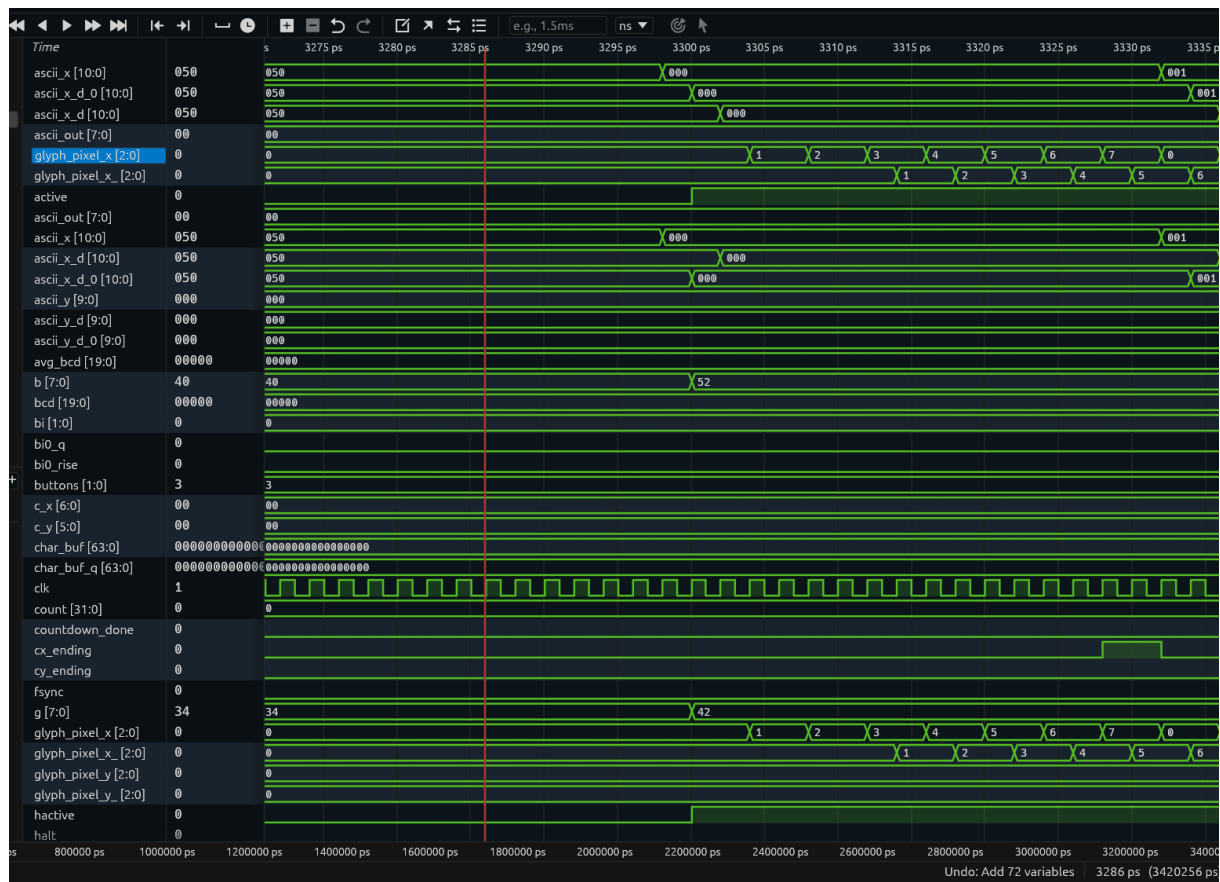
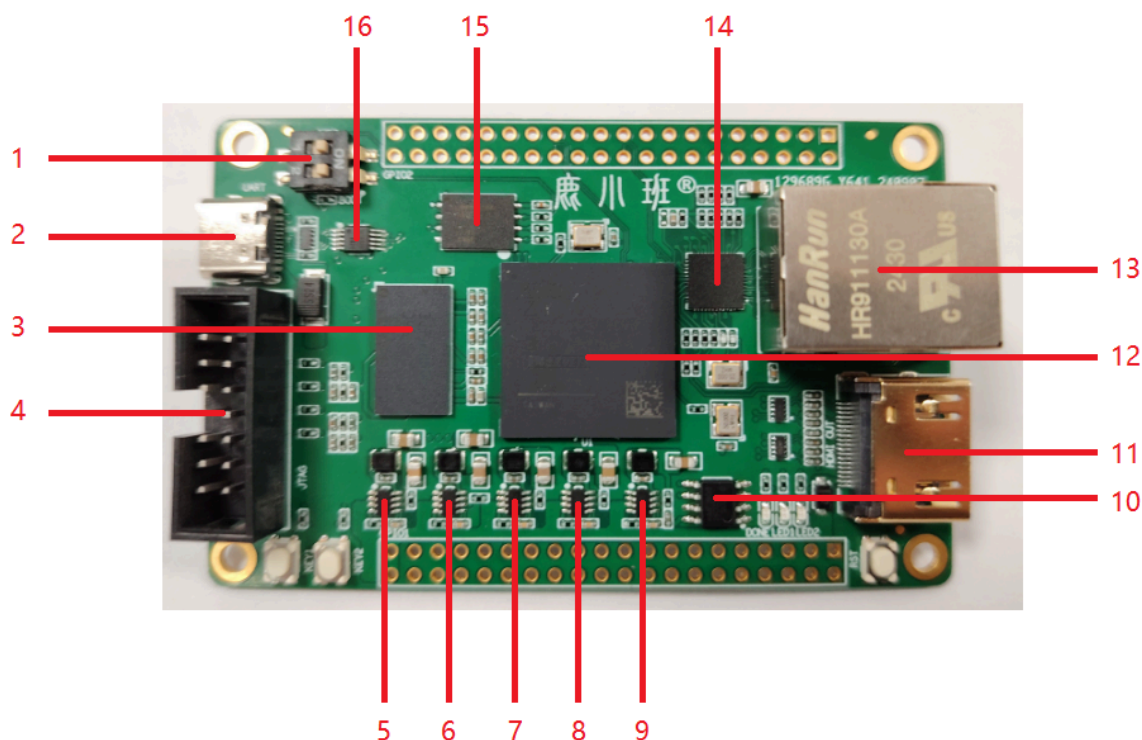


图 7 top_simulation 测试波形图: vga 输出 pipeline

以上测试波形图可采用 just run_verbose 生成，并通过 just surfer 打开 surfer 监视波形图。

4.5 FPGA 板介绍

本次实验采用了 鹿小班 ZYNQ7020 开发板: <https://gitee.com/GLSZ/LXB-ZYNQ>



4.6 综合与实现

4.6.1 综合条件

使用 Vivado 2025.2 进行综合与实现，目标器件为 xc7z020clg484-2。综合后资源利用率如下。

4.6.2 资源利用率

Type	Used	Available	Utilisation
Slice LUTs	1095	53200	2.06%
Slice Registers	525	106400	0.49%
Block RAM	1	140	0.71%
DSP48E1	0	220	0%
Bonded IOB	13	200	6.5%
BUFGCTRL	3	32	9.38%
PLLE2_ADV	1	4	25%
OSERDESE2	8	200	4%

4.6.3 时序结果

Metric	Value
Worst Negative Slack (WNS)	1.157 ns
Total Negative Slack (TNS)	0 ns
Worst Hold Slack (WHS)	0.116 ns
Total Hold Slack (THS)	0 ns
Worst Pulse-Width Slack	1.074 ns

Clock Summary

Clock	Period	Frequency
clk50	20 ns	50 MHz
clk_75	13.333 ns	75 MHz
clk_375 (SERDES)	2.667 ns	375 MHz
PLL feedback	40 ns	25 MHz

通过计算

$$F_{\max} = \frac{1000}{\text{Target Period} - \text{WNS}} = \frac{1000}{13.33 - 1.157} = 82.129 \text{ MHz}$$

4.6.4 功耗

Metric	Value
Total On-Chip Power	0.364 W
Dynamic Power	0.254 W
Device Static Power	0.11 W
Junction Temperature	29.2 °C

4.7 芯片物理实现

使用 Vivado 2025.2 完成物理实现，包含自动布局布线、时钟树综合、以及 bitstream 生成。总片上功耗 0.371 W，结温 29.3 °C。

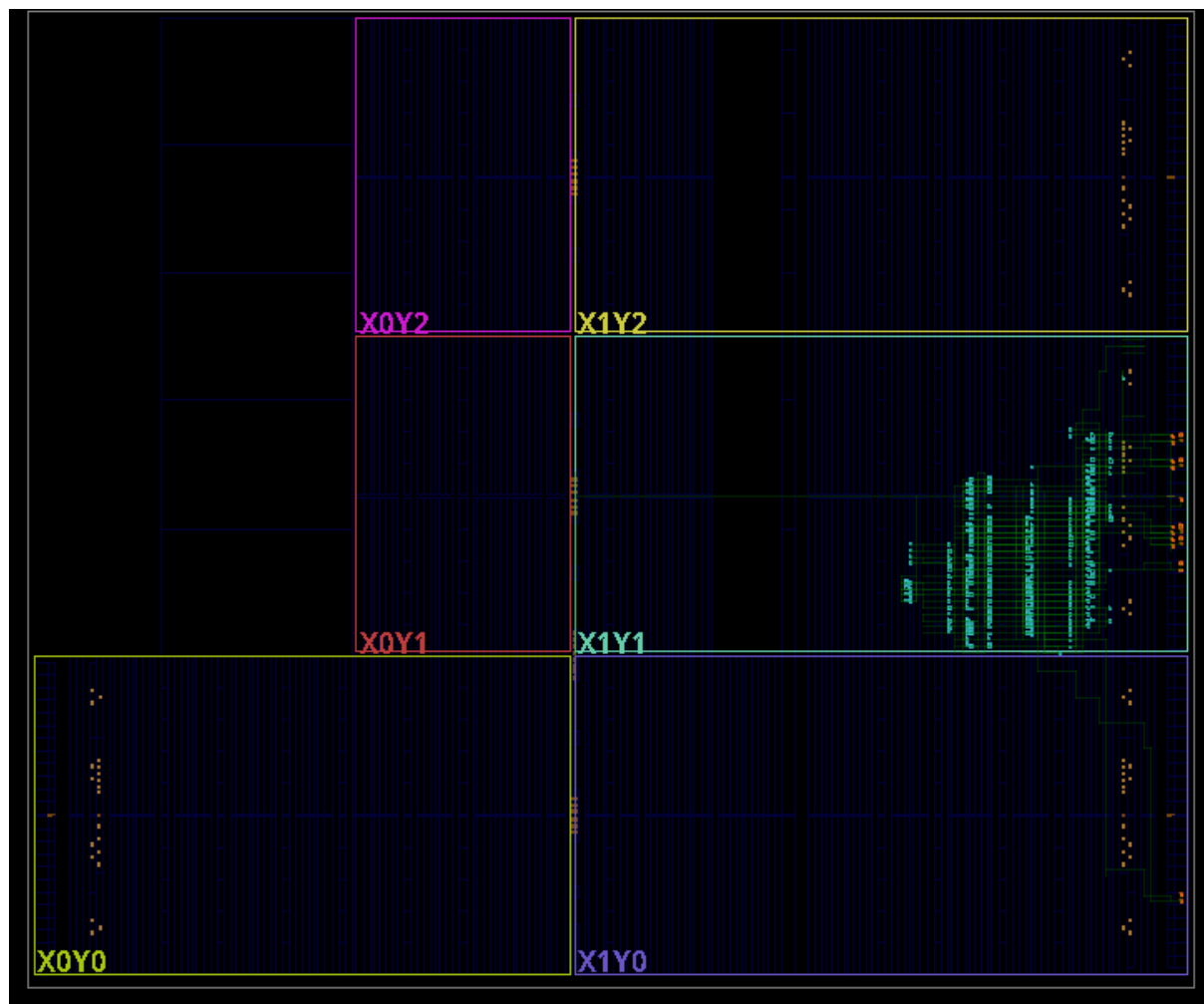


图 9 Implemented design for FPGA in vivado

为了体验一下 ASIC 的自动化流程，本人还尝试在 opencode 的加持下把原有的 sv 转译成 yosys 可读取的 sv 格式(比较旧，不支持 dynamic string)，然后成功利用 librelane 生成了芯片 GDS2 Layout 如下：

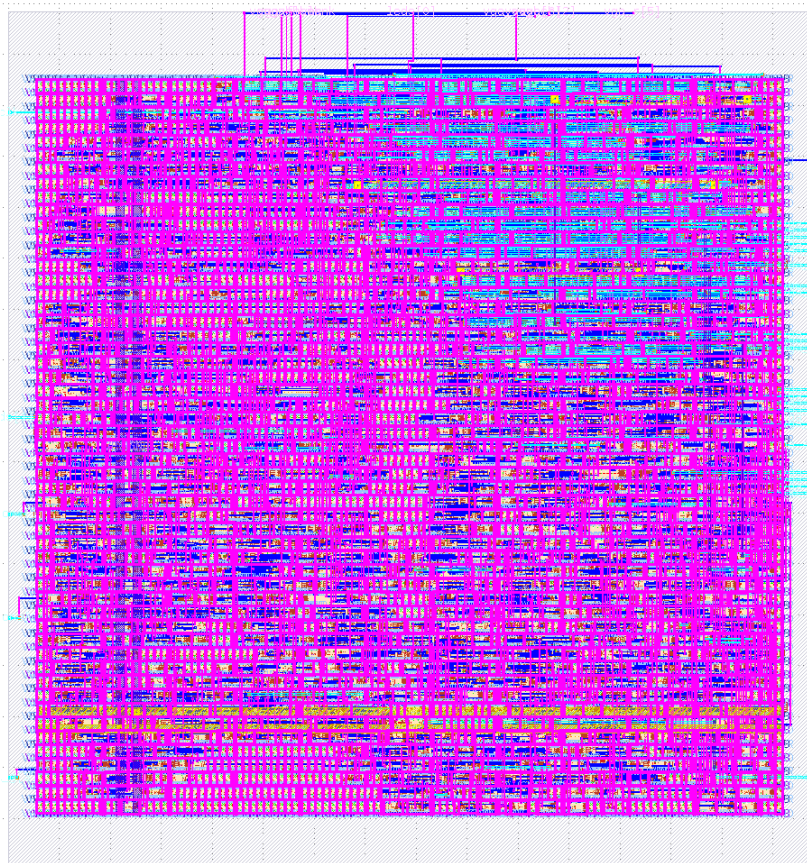


图 10 利用 librelane 生成的 layout (sky130 工艺库)



图 11 利用 librelane 生成的 layout (sky130 工艺库) vga 部分端口

注意：上述 layout 是 top_vga 模块的，并非 top。

4.8 上板结果展示

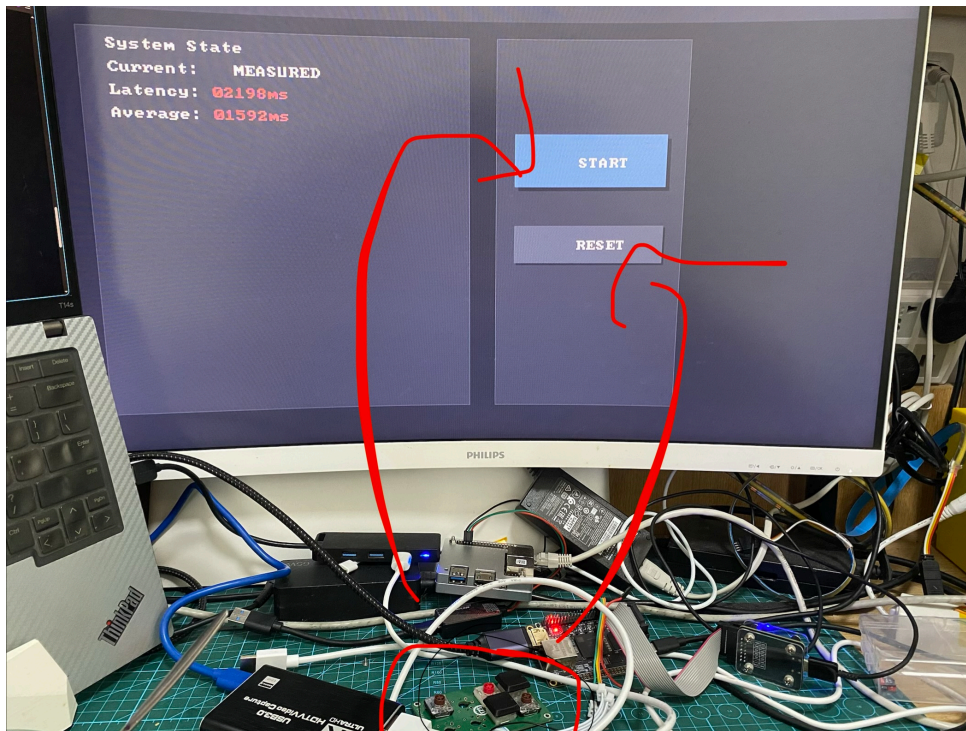


图 12 实际效果，红箭头为按键对应输入

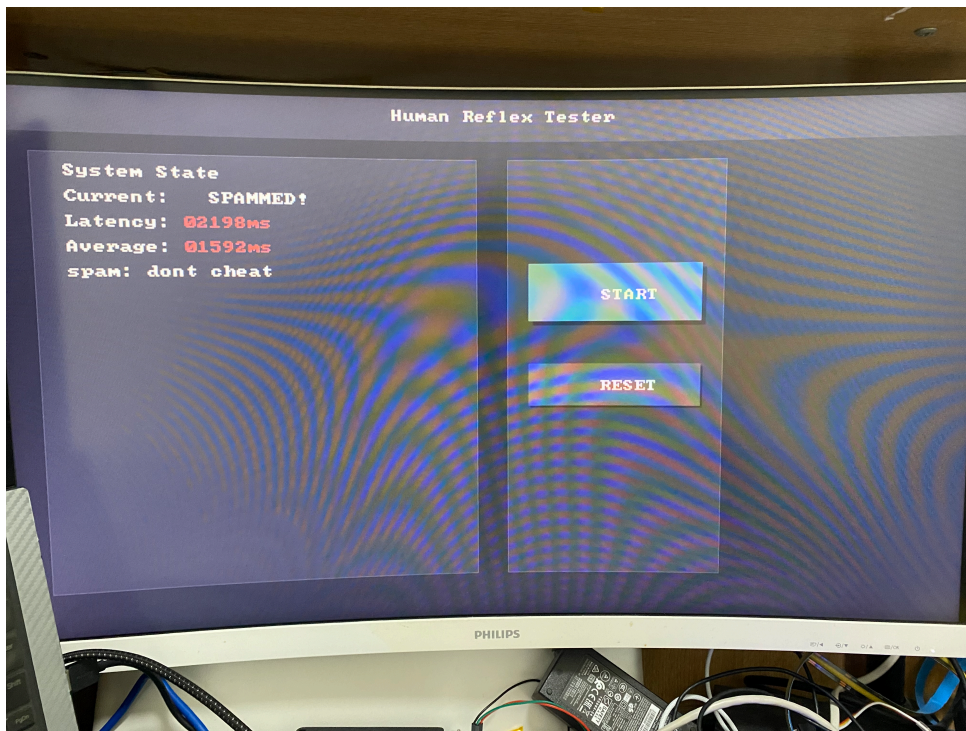


图 13 如果用户尝试欺骗系统，系统将显示 SPAM，并终止程序

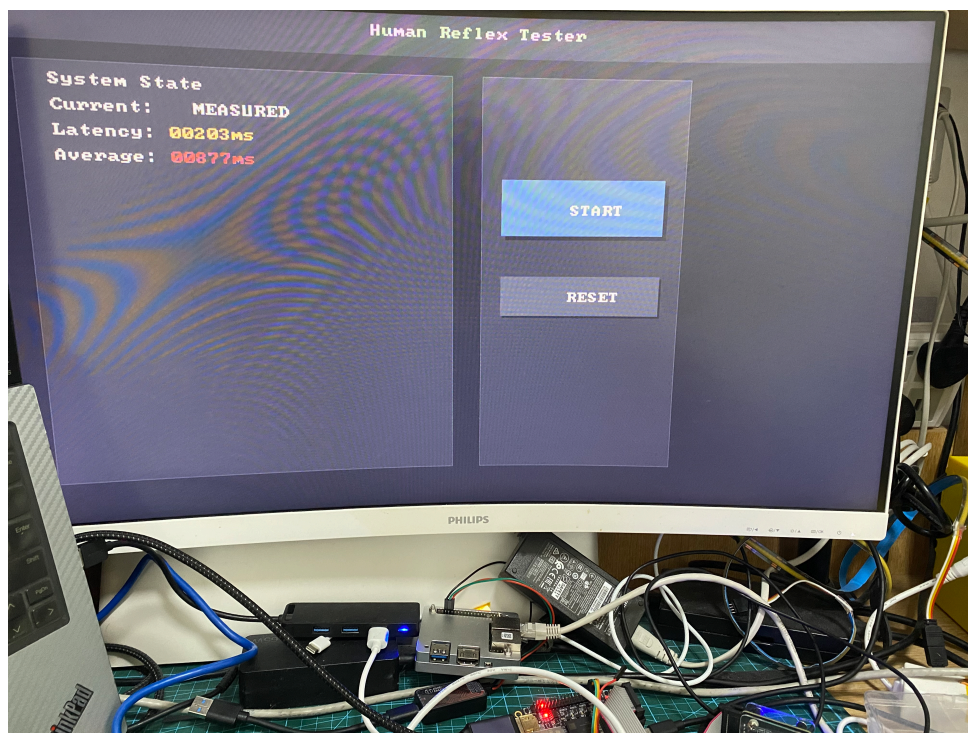


图 14 延迟根据实际范围改变颜色 (靠近绿色则表示健康, 靠近红色则表示不健康)

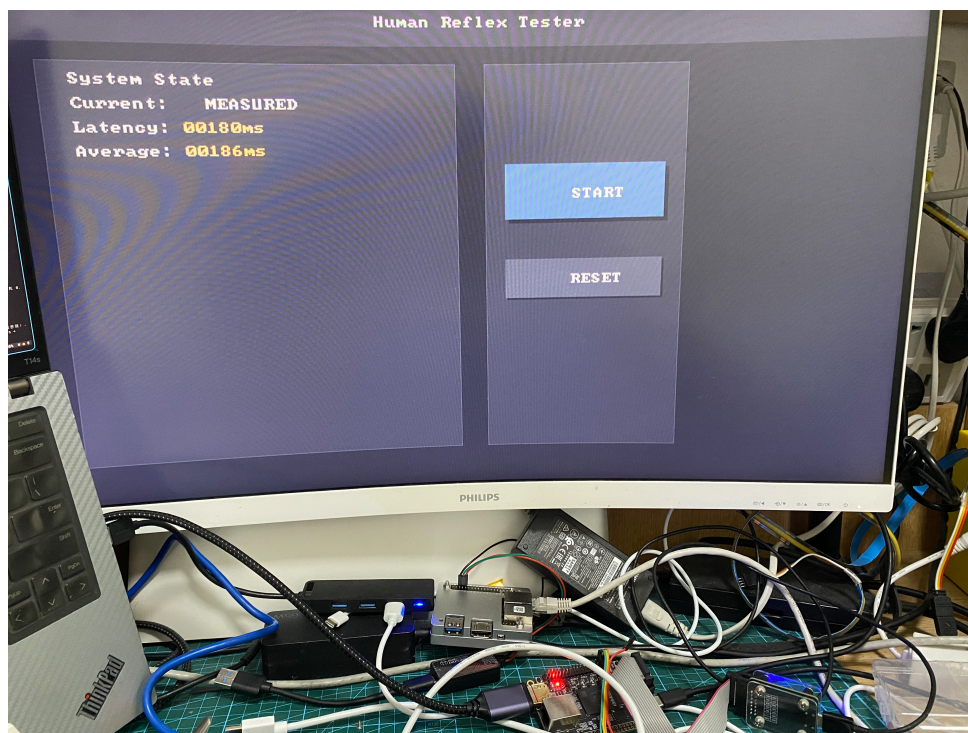


图 15 延迟根据实际范围改变颜色 (靠近绿色则表示健康, 靠近红色则表示不健康)

4.9 设计总结

本设计实现了基于 FPGA 的人体反应速率测试系统, 覆盖了按键消抖、随机延时生成、高精度计时 (毫秒级)、滑动平均滤波、防作弊检测、HDMI 显示等完整功能链。全流程使用开源工具链 (Verilator + vivado + openFPGALoader) 配合自定义 Rust 仿真器进行开发验证。

4.10 个人体会

这次的 project 实际上并不难, 其中花最长时间的在于优化整个工具链。起初, 我尝试利用 rust 来实现整个测试流程, 使用的 marlin 库。但由于其生态还未完全成熟 (比如, 测试不支持直接调用 sv enum), 因此在测试的过程中遇到很多节外生枝的小问题。然而, 我看到的是利用 rust (marlin) 开发测试的确比 Chisel、cocotb 来的好。原因在于因为 rust 本身的代码库的生态做的非常棒, 可以做到高度的 integration, 从而使得整个开发过程变得更加简洁。代码简洁的目的其实就是为了规避掉很多低级错误, 避免过度重复不必要的工作, 同时也可以降低程序员的负荷。同时, 我也探索了 spade 语言等新型 HDL, 由于 Spade 过于新, 处于不稳定周期, 所以我暂时不敢常采用它来开发此次项目。但我自己用下来感觉 spade 应该是未来的方向。原因有很多, 其中以 pipeline 自带功能确实能够让整个开发变得更加容易, 而且不易出错。这让我意识到, 简洁性其实不是只是为了美观, 而是让自己减轻思考的负荷, 让开发流程变得更鲁棒。

Spade is a hardware description language inspired by modern software languages.

- Strong and expressive **type system** to catch bugs early
- First-class **pipelines**
- Zero-cost **Abstractions** for hardware
- **Helpful compiler** with great error messages
- **Cute mascot**

```
// Compute the dot product of two arrays of any size
pipeline(3) dot_product<#uint N>(
  clk: clock,
  rst: bool,
  a: [int<16>; N],
  b: [int<16>; N],
) -> int<32> {
  let products = a
    .zip(b)
    .map(fn |(a, b)| a * b)
  reg * 3;
  // Return the sum of the products
  products.sum()
}
```

Curious about a keyword in the examples? Click on it to learn more

以前写过 VGA, 这次换成写 HDMI 才发现, 原来 HDMI 是基于 DVI, 并在其基础上引入额外的信息传输, 所以使用下来感觉还是蛮友好的。

最后, 我感觉 Vivado 对编译错误不敏感的这个默认行为真的会害死很多开发者。有时候因为一些低级错误被视为 Warning, 所以容易被忽视, 结果是浪费了一整天的时间在寻早 bug 的源头。当然 git 可以帮助你快速挑出错误点, 但依然是治标不治本啊。